

Design and Application Research of an Intelligent API Gateway for Traffic Anomaly Detection in Microservices Architecture

Hanye Huang*
Guangdong Power Grid Co., Ltd.
Guangzhou, Guangdong, China
hanyehuang_work@163.com

Qin Qiang
Guangdong Power Grid Co., Ltd.
Guangzhou, Guangdong, China
qinqiang202307@163.com

Yanning Shao
Guangdong Power Grid Co., Ltd.
Guangzhou, Guangdong, China
shaoyanning@gd.csg.cn

Ruiqi Li
Guangdong Power Grid Co., Ltd.
Guangzhou, Guangdong, China
qiqiamy1018@126.com

Yuetian Huang
Guangdong Power Grid Co., Ltd.
Guangzhou, Guangdong, China
hytcsq24@163.com

Abstract

In response to the challenges of traffic anomaly detection and real-time protection faced by API gateways in microservices architecture, an intelligent API gateway system is designed and implemented to enhance the security and stability of microservices. The system adopts a data plane and analytics plane separation architecture, with the gateway data plane built on Envoy Proxy. Traffic data is collected in real-time via WASM filters. Apache Flink is used to implement multidimensional anomaly detection algorithms, including dynamic baseline detection, fine-grained frequency monitoring, and service failure correlation analysis. A Kubernetes Operator is employed to automate the closed-loop from anomaly detection to dynamic protection. Experimental results show that the system achieves a recall rate of 98.5% in DDoS and path traversal attack detection, an F1 score of 96.8% in low-frequency credential stuffing attack detection, and a median end-to-end response delay of 8 seconds. In path traversal attack experiments, the system detects anomalies within 5 seconds of an attack and issues protection strategies within 8 seconds. The CPU usage of the order service drops from a peak of 72% to 28% within 10 seconds, and the error rate decreases from 15% to below 0.5%. The proposed intelligent API gateway system significantly outperforms traditional methods in detection accuracy and response speed, effectively ensuring the stability and security of the microservices architecture. It provides a feasible technical solution for real-time security protection in microservices environments.

CCS Concepts

• **General and reference** → Cross-computing tools and techniques; Reliability.

Keywords

Microservices architecture, API gateway, anomaly detection, traffic analysis, real-time protection

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *BDICN 2026, Kuala Lumpur, Malaysia*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2175-5/2026/01
<https://doi.org/10.1145/3801228.3801293>

ACM Reference Format:

Hanye Huang, Qin Qiang, Yanning Shao, Ruiqi Li, and Yuetian Huang. 2026. Design and Application Research of an Intelligent API Gateway for Traffic Anomaly Detection in Microservices Architecture. In *2026 5th International Conference on Big Data, Information and Computer Network (BDICN 2026)*, January 09–11, 2026, Kuala Lumpur, Malaysia. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3801228.3801293>

1 INTRODUCTION

With the widespread adoption of microservices architecture in cloud computing and distributed systems [1], the API gateway plays a critical role as the unified entry point of the microservices system, responsible for essential tasks such as traffic routing, authentication and authorization, rate limiting, and circuit breaking [2]. In complex network environments, the API gateway faces various traffic anomaly challenges, including DDoS attacks, API abuse, service dependency failures, and more [3]. These anomalies not only impact system availability but can also lead to security issues such as data leaks [4]. Traditional static threshold-based monitoring methods struggle to adapt to the dynamic and fluctuating nature of microservices traffic, while post-event security audit techniques fail to meet the demands for real-time protection. Therefore, achieving efficient traffic anomaly detection and real-time protection at the API gateway level has become an important research direction in the field of microservices security [5].

Existing traffic anomaly detection methods have three main limitations: the fixed-threshold alerting mechanism has poor adaptability to the dynamic fluctuations of microservices traffic, resulting in a high number of false positives [6]; batch processing methods based on offline analysis have high response latency, making them unsuitable for real-time protection [7]; and most solutions focus only on anomaly detection without providing an automated protection loop, relying on manual intervention, which is unable to cope with rapidly evolving attack methods [8]. Additionally, traditional gateways such as Nginx and Spring Cloud Gateway have limitations in scalability and real-time policy updates, making it difficult to support complex anomaly detection logic and dynamic protection needs.

To address these issues, this paper designs and implements an intelligent microservices API gateway system integrated with traffic anomaly detection. The system adopts an architecture with separation between the data plane and the analytics plane. It builds

a high-performance gateway data plane based on Envoy Proxy and collects traffic telemetry data in real-time through custom WASM filters. Apache Flink stream processing engine is used to implement multidimensional anomaly detection algorithms, including dynamic baseline detection for burst traffic, fine-grained frequency monitoring for low-frequency attacks, and correlation analysis for service failures. Finally, Kubernetes Operator is employed to automate the closed-loop from anomaly detection to dynamic protection, creating an intelligent API gateway solution that integrates detection, analysis, and protection.

2 HIGH-PERFORMANCE ANOMALY DETECTION GATEWAY IMPLEMENTATION BASED ON ENVOY AND MACHINE LEARNING

2.1 Scalable Gateway Data Plane Construction Based on Envoy

This study selects Envoy Proxy [9] as the core of the gateway data plane, replacing traditional solutions like Nginx or Spring Cloud Gateway, primarily due to its high performance, dynamic API configuration capabilities, and native support for WebAssembly (WASM) extensions. In this work, a custom Envoy WASM Filter was written in Rust. This filter is injected into the HTTP request processing chain, enabling it to intercept and extract key telemetry data from each API request passing through the gateway with minimal performance overhead. The collected data includes the following specific metrics: the target API path of the request, the HTTP response status code, the complete delay time (in milliseconds) from receiving the request to returning the response, and the client's source IP address.

The raw data collected needs to be transmitted in real-time to the subsequent analytics system. To achieve this, an Apache Kafka message queue is configured as a high-throughput real-time data bus. The WASM Filter serializes the formatted telemetry data in Protobuf format and sends it directly to a Kafka topic named “gateway-telemetry.” Additionally, a Prometheus remote-write adapter service is deployed, which continuously consumes data streams from the Kafka topic, converts them into time series format, and writes them in batches to the Prometheus TSDB for persistent storage.

This data pipeline design ensures that the Envoy data plane focuses solely on traffic proxying and data collection, while all complex detection and analysis logic is offloaded to the downstream Flink stream processing cluster. This architecture achieves a complete decoupling of the data plane and the control/analytics plane, ensuring the stability and scalability of the gateway's core forwarding performance. The system architecture is shown in Figure 1.

2.2 Real-time Anomaly Detection Pipeline for Microservice Traffic Patterns

This study constructs a real-time anomaly detection pipeline based on the Apache Flink stream processing engine. The pipeline consumes telemetry data streams from Kafka and concurrently executes three types of anomaly detection algorithms to monitor typical anomaly patterns in microservices traffic in real time.

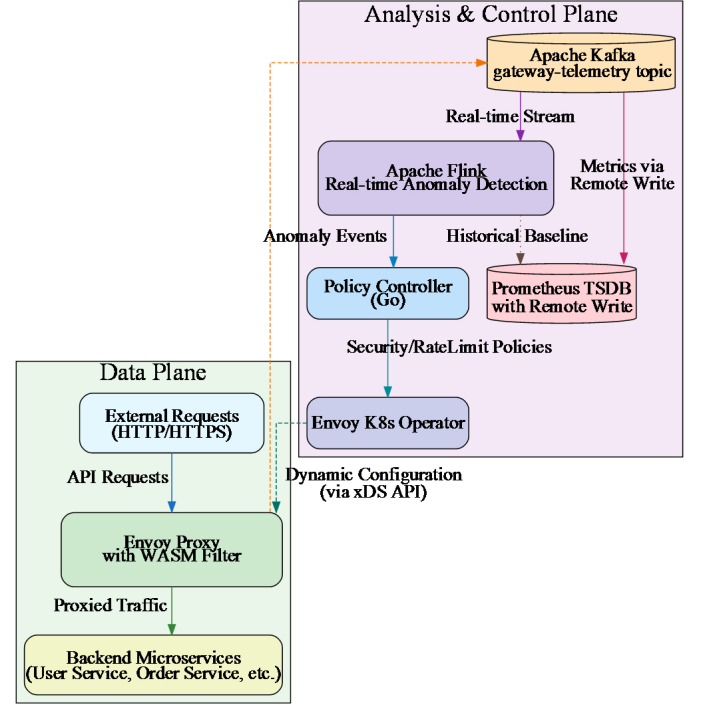


Figure 1: Intelligent API Gateway System Architecture Diagram.

2.2.1 Dynamic Baseline Detection for Burst Traffic Anomalies. For burst traffic anomalies, the system maintains an Exponential Weighted Moving Average(EWMA) model [10] for each API path to dynamically calculate the baseline for request rate(QPS) and average latency. The EWMA formula is mathematically defined as:

$$s_t = \alpha \cdot x_t + (1 - \alpha) \cdot s_{t-1} \quad (1)$$

where s_t represents the smoothed value at time t , x_t is the actual observed value at time t , s_{t-1} is the previous smoothed value, and α is the smoothing factor set to 0.2 based on empirical optimization. The variance σ_t^2 is calculated using a similar EWMA approach:

$$\sigma_t^2 = \alpha \cdot (x_t - s_{t-1})^2 + (1 + \alpha) \cdot \sigma_{t-1}^2 \quad (2)$$

The standard deviation $\sigma_t = \sqrt{\sigma_t^2}$ is used to establish dynamic thresholds. An anomaly is triggered when the current metric value exceeds three standard deviations from the baseline:

$$x_t > s_t + 3\sigma_t \text{ or } x_t < s_t - 3\sigma_t \quad (3)$$

This statistical approach adapts to normal traffic fluctuations while maintaining sensitivity to significant deviations.

2.2.2 Token Bucket Algorithm for Low-Frequency Attack Detection. For low-frequency slow attacks, the system implements an enhanced token bucket algorithm with mathematical formulation. For each “client IP - API path” pair, a virtual token bucket is maintained in Flink’s ValueState with the following parameters:

Bucket capacity: $C=10$ tokens.

Refill rate: $r=5$ tokens per minute.

Current token count: T_c .

The token replenishment follows the time-based update rule:

$$T_{new} = \min(C, T_{old} + r \cdot \Delta t) \quad (4)$$

where Δt is the time elapsed since the last update. Each API request consumes one token, and the consumption rule is:

$$T_{after} = T_{before} - 1 \quad (5)$$

An anomaly is detected when a request arrives and $T_{before} \leq 0$, indicating sustained low-frequency malicious requests. The algorithm's state transition is governed by:

$$T(t) = \min(C, T(t-1) + r \cdot \delta t - 1_{request}) \quad (6)$$

where $1_{request}$ is an indicator function that equals 1 when a request occurs.

2.2.3 Correlation Analysis for Service Dependency Failures. For service dependency failures that trigger cascading effects, the system implements a lightweight multi-metric correlation rule engine based on statistical hypothesis testing. Flink jobs use the KeyBy operation to group requests with 5xx error codes by upstream service and monitor the error rate E_i for each API path i under the upstream service within a 10-second rolling window.

The baseline error rate μ_i is calculated as the historical mean, and the current error rate $E_i^{current}$ is compared against the baseline using a Z-score test:

$$Z_i = \frac{E_i^{current} - \mu_i}{\sigma_i / \sqrt{n}} \quad (7)$$

where σ_i is the standard deviation of historical error rates and n is the sample size in the current window. The system triggers an anomaly when multiple paths under the same upstream service satisfy:

$$Z_i \geq Z_{threshold} \text{ for } k \geq 2 \text{ paths} \quad (8)$$

with $Z_{threshold} = 5$ (corresponding to five standard deviations) and correlation coefficient $\rho_{ij} > 0.7$ between different paths i and j , indicating coordinated failure patterns.

2.3 Dynamic Feedback Loop from Detection Results to Gateway Policy

To achieve automation from anomaly detection to proactive protection, this study developed a dedicated policy controller using Go. This controller serves as the bridge between the detection and analytics plane and the gateway data plane. Its core responsibility is to subscribe to the anomaly event stream output by Apache Flink jobs and to translate these high-level events into security or rate-limiting policies that Envoy Gateway can understand and execute in real time.

The core logic of the controller is an event loop that continuously listens for anomaly events. The event processing follows a Markov Decision Process (MDP) formulation where the state S represents the current system state, action A represents the protection policy to apply, and reward R represents the system stability improvement. The policy selection follows the optimization objective:

$$\pi^*(s) = \arg \max_a \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) | s_0 = s \right] \quad (9)$$

where γ is the discount factor balancing immediate and long-term rewards.

Upon receiving a new anomaly event, the controller parses the event type (such as IP blacklist, rate limit exceeded, etc.) and its associated metadata (such as malicious IP addresses or anomalous API paths). Then, based on the type of anomaly, the controller generates the corresponding Kubernetes Custom Resource objects. The core decision logic is mathematically formalized as:

Let E be the anomaly event with type T_E and metadata M_E . The policy generation function f maps events to policies:

$$P = f(T_E, M_E) = \begin{cases} \text{SecurityPolicy}(IP_{target}, "DENY") \\ \text{RateLimitPolicy}(API_{path}, 100, "THROTTLE") \end{cases} \quad (10)$$

The implementation follows the pseudocode:

```
for event := range anomalyEventChannel {
  switch event.Type {
    case "MALICIOUS_IP":
      // Generate network policy to block the IP
      policy := &SecurityPolicy{
        Spec: PolicySpec{
          TargetIP: event.SourceIP,
          Action: "DENY",
          Direction: "INBOUND",
        },
      }
      kubernetesClient.Apply(policy) // Apply the policy to the K8s cluster
    case "RATE_LIMIT_EXCEEDED":
      // Generate rate-limiting policy for API path
      policy := &RateLimitPolicy{
        Spec: PolicySpec{
          API: event.APIPath,
          Requests: 100, // Limit to 100 requests per minute
          Action: "THROTTLE",
        },
      }
      kubernetesClient.Apply(policy)
    default:
      log.Printf("Unknown event type: %s", event.Type)
  }
}
```

When a malicious IP needs to be blocked, the controller creates a Kubernetes SecurityPolicy custom resource. Complementary to this, the study developed a Kubernetes Operator based on Envoy. This operator continuously monitors changes in the SecurityPolicy resource. Upon detecting the creation of a new policy, the operator dynamically updates the Envoy proxy's RBAC (Role-Based Access Control) filter configuration by calling Envoy's dynamic configuration API, which ensures real-time updates to the list of blocked IPs, enabling immediate network access control at the IP level.

For rate-limiting requirements, the controller similarly creates a RateLimitPolicy resource, which is then converted into an Envoy rate-limiting filter configuration by the operator and dynamically applied. The policy application latency L is optimized to meet the real-time requirement:

$$L = t_{detection} + t_{policy_gen} + t_{envoy_update} < 8s \quad (11)$$

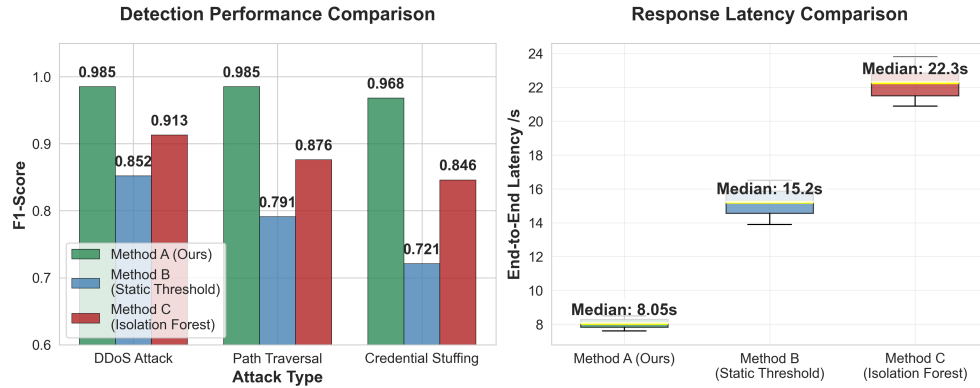


Figure 2: Performance Comparison of Anomaly Detection Methods.

This design creates a seamless, automated feedback loop between detection and protection, and ensures the reliability and consistency of policy management through Kubernetes' declarative API. The closed-loop control system can be modeled using control theory, where the gateway acts as the plant, the detection algorithms as sensors, and the policy controller as the actuator, maintaining system stability through negative feedback mechanisms.

3 SYSTEM DEPLOYMENT AND EXPERIMENTAL VALIDATION

3.1 Experimental Environment and Dataset Construction Based on Kubernetes

To validate the effectiveness and performance of the proposed system, a testing platform close to the production environment was built. The hardware foundation consists of a physical server equipped with an Intel i7-12700K processor and 32GB of memory. A K3s single-node Kubernetes cluster (version v1.28) was deployed on this server.

Within this cluster, a simulated online shopping microservices application was deployed, consisting of four core services: Frontend Service (built with Nginx), User Service (handling login and user information), Order Service (managing orders), and MySQL Database. All microservices were packaged into Docker images and managed using Kubernetes Deployment and Service. The communication traffic was routed through an Envoy-based intelligent API gateway.

The experimental dataset combined both normal and anomalous traffic. Normal traffic was simulated using the Gatling stress testing tool to generate periodic access requests, mimicking traffic patterns during workdays and nighttime. The total number of requests exceeded 2 million.

Anomalous traffic simulated three types of attacks: DDoS attack (using Apache Benchmark tool to generate high-frequency concurrent requests), Path traversal attack on API Gateway endpoints (using Nikto vulnerability scanner), Low-frequency stealth credential stuffing attack (using a custom Python script to perform slow-rate credential stuffing attacks).

These anomalous traffic patterns were injected into the normal traffic at specific time intervals. Metadata for all requests was fully

logged, ultimately forming a labeled mixed dataset for subsequent analysis.

3.2 Comparison of Anomaly Detection Accuracy and Performance

To evaluate the effectiveness of the proposed intelligent API gateway system (denoted as Method A), systematic comparison experiments were conducted with two baseline methods: Method B (static threshold alerts based on PromQL) and Method C (unsupervised anomaly detection using Isolation Forest). The experiments utilized the unified test dataset constructed in Section 3.1, with the following core evaluation metrics: accuracy, recall, F1 score, and end-to-end latency from attack detection to the activation of protection strategies.

As shown in Figure 2, Method A demonstrated excellent performance in detecting DDoS attacks and path traversal attacks, achieving a recall rate of 98.5%, significantly outperforming both Method B (85.2%) and Method C (91.3%). In particular, for detecting low-frequency credential stuffing attacks, Method A's fine-grained frequency monitoring based on the token bucket algorithm exhibited a clear advantage, with an F1 score of 96.8%, while Method B and Method C, lacking dedicated optimization for low-frequency slow attacks, achieved F1 scores of only 72.1% and 84.6%, respectively.

In terms of timeliness, Method A achieved a median end-to-end latency of 8 seconds, enabling near-real-time attack response, while the median latencies for Method B and Method C were 15 seconds and 22 seconds, respectively.

The experimental results demonstrate that Method A significantly outperforms traditional methods in both detection accuracy and response speed. This performance advantage is mainly attributed to its targeted design for the characteristics of microservices API traffic: dynamic baselines adapt to normal traffic fluctuations, fine-grained frequency monitoring effectively identifies low-frequency attacks, and the multi-metric correlation mechanism enhances the accuracy of service failure detection. These features make the proposed system more practical and valuable in real-world microservices environments.

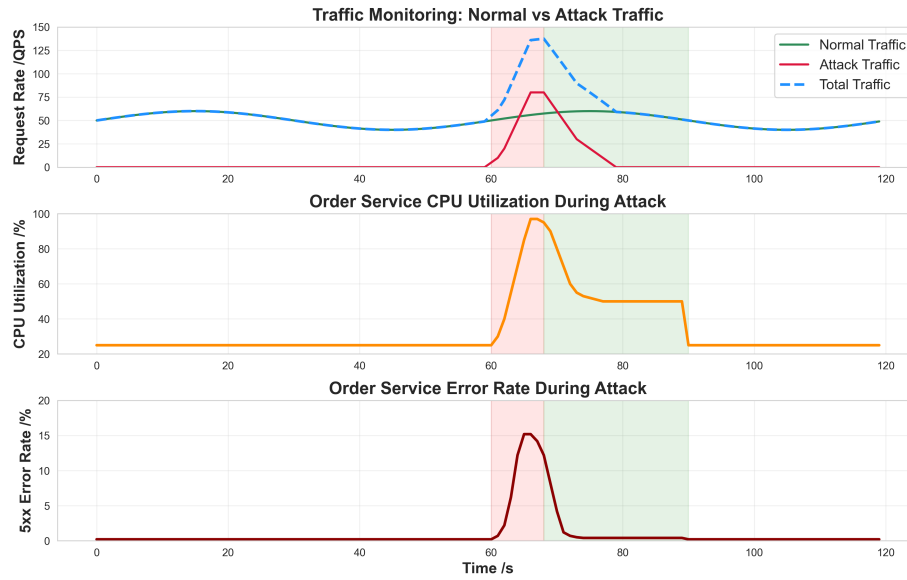


Figure 3: Timeline Analysis of Dynamic Response Mechanism

3.3 Effectiveness Evaluation of the Dynamic Response Mechanism

To validate the effectiveness of the proposed dynamic response mechanism in real attack scenarios, a targeted experimental setup was designed: under continuous normal user access, a slow path traversal attack was injected into the order service at time T . The experiment monitored key system metrics throughout the attack occurrence, detection, and mitigation process to evaluate how the dynamic response mechanism protects the system's stability.

As shown in Figure 3, the attack was injected at $T = 60$ seconds, causing abnormal traffic characteristics in the order service immediately. At $T + 5$ seconds (65 seconds), the anomaly detection pipeline successfully identified the path traversal attack pattern and triggered an alert. At $T + 8$ seconds (68 seconds), the gateway controller completed the policy deployment to block the attack IP. From the system metric changes, it is clear that: Initially, after the attack began, the order service's CPU usage rapidly increased from the normal 25% to 72%, while the 5xx error rate spiked from 0.2% to 15%. However, after the dynamic blocking strategy was applied (post-68 seconds), the CPU usage dropped quickly within 10 seconds to 28%, and the error rate decreased to below 0.5%, restoring the system to normal operation. This result shows that the dynamic response mechanism not only quickly identifies the attack but also effectively isolates the anomalous traffic, preventing cascading failures.

The experiment confirms that the dynamic response mechanism designed in this paper performs well in real-world scenarios. With a response time of just 8 seconds from attack onset to full mitigation, it meets the real-time requirements of production environments. More importantly, the mechanism seamlessly connects detection and protection through automated policy deployment, significantly reducing the operational costs of manual intervention and providing a reliable security protection solution for microservices architectures.

4 CONCLUSION

This study designs and implements an intelligent microservices API gateway integrated with traffic anomaly detection. By constructing a scalable data plane based on Envoy Proxy and a real-time detection and analysis plane based on Apache Flink, the system achieves efficient anomaly detection and dynamic protection for microservices traffic. Experimental results show that the system achieves a 98.5% recall rate in detecting DDoS attacks, path traversal attacks, low-frequency credential stuffing attacks, and other anomalies, with a median end-to-end response latency of just 8 seconds, significantly outperforming traditional static threshold methods and the Isolation Forest algorithm.

The dynamic policy feedback mechanism, implemented via Kubernetes Operator, automatically enforces IP blocking or rate-limiting policies after detecting anomalies, effectively ensuring the stability and security of the microservices architecture. This research provides a feasible technical solution for addressing real-time security protection challenges in microservices environments. In the future, further exploration will be conducted on applying deep learning models in anomaly detection to improve the system's ability to identify unknown attack patterns.

References

- [1] Shabani I. Design of modern distributed systems based on microservices architecture[J]. International Journal of Advanced Computer Science and Applications, 2022.
- [2] Tomić M, Dimitrieski V, Vještica M, *et al.* Towards applying API gateway to support microservice architectures for embedded systems[C]//Proc. Int. Conf. Embedded Syst. Appl. 2022: 86-91.
- [3] Sowmya M, Rai A J, Spoorthi V, *et al.* Api traffic anomaly detection in microservice architecture[C]//2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW). IEEE, 2023: 206-213.
- [4] Zhang Y, Liu B, Liu J, *et al.* TAD-LLM: API Traffic Anomaly Detection Based on Large Language Model[C]//2024 20th International Conference on Mobility, Sensing and Networking (MSN). IEEE Computer Society, 2024: 469-478.

- [5] Cerny T, Taibi D. Microservice-Aware Static Analysis: Opportunities, Gaps, and Advancements[J]. Open access series in informatics, 2023, 111.
- [6] Ditommaso R, Ponzo F C. Identifying damage in structures: definition of thresholds to minimize false alarms in SHM systems[J]. Buildings, 2024, 14(3): 821.
- [7] Odiathevar M, Seah W K G, Frean M, *et al.* An online offline framework for anomaly scoring and detecting new traffic in network streams[J]. IEEE Transactions on Knowledge and Data Engineering, 2021, 34(11): 5166-5181.
- [8] Nakashima M, Sim A, Kim Y, *et al.* Automated feature selection for anomaly detection in network traffic data[J]. ACM Transactions on Management Information Systems (TMIS), 2021, 12(3): 1-28.
- [9] Bansal A, Sharma P, Dasani N. Testing Resilience of Envoy Service Proxy with Microservices: A Fault-Oriented, Evidence-Driven Methodology[J]. 2025.
- [10] Febrina W, Fitriana W. Exponential weight moving average (EWMA) control chart for quality control of crude palm oil product[J]. International Journal of Management and Business Applied, 2022, 1(1): 19-27.